

Amélioration du livrable P6 grâce à l'IA

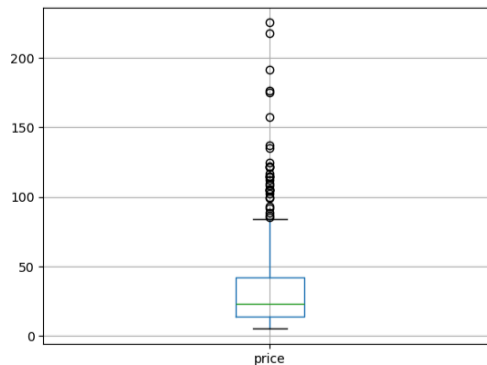
Documentation de la démarche — veille métier et technologique, cahier des charges fonctionnel, mini-formation, gestion de projet

Améliorez le livrable du P6 grâce à l'IA

Pour ce module, j'ai choisi d'améliorer l'étape 4.1 du P6. Actuellement il y a deux fois le même graphique : une boîte à moustaches avec pandas, puis la même chose avec plotly. Ça montre la médiane/les quartiles/les valeurs aberrantes, mais pas la forme de la distribution (asymétrie, concentration des prix, etc.). C'est un axe d'amélioration simple et concret.

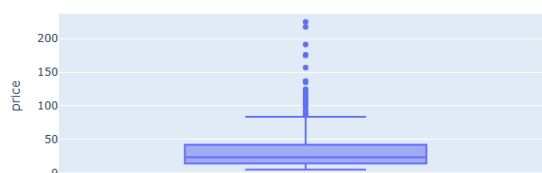
```
: #Création d'une boîte à moustache de la répartition des prix grâce à Pandas
df_merge_web_clean.boxplot(column='price')
```

: <Axes: >



```
: #Autre méthode avec plotly express
fig = px.box(df_merge_web_clean, y='price', title="Répartition des prix")
fig.show()
```

Répartition des prix



État initial — boxplot en double (pandas puis plotly)

Prompt A : Dans ce notebook, la distribution des prix est visualisée uniquement avec un boxplot (pandas puis plotly, en double). Propose une visualisation plotly express qui montre à la fois la distribution complète des prix et les valeurs aberrantes.

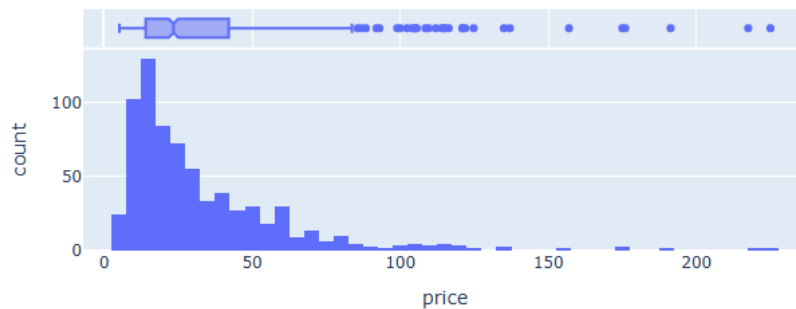
Code obtenu

```
python

fig = px.histogram(df_merge_web_clean, x='price', marginal='box',
                   title="Distribution des prix (avec repérage des outliers)")
fig.show()
```

Résultat obtenu

Distribution des prix (avec repérage des outliers)



Prompt B : Propose une alternative avec la librairie seaborn pour visualiser en un seul graphique la distribution des prix et les valeurs aberrantes.

Code obtenu

```
python

import seaborn as sns
import matplotlib.pyplot as plt

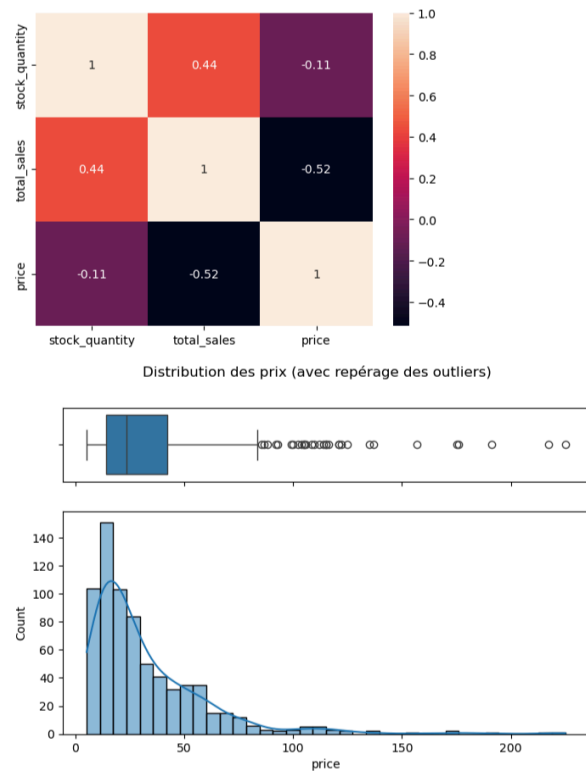
fig, axes = plt.subplots(2, 1, figsize=(8,5), sharex=True, gridspec_kw={'height_ratios': [1,3]})
sns.boxplot(x=df_merge_web_clean['price'], ax=axes[0])
sns.histplot(x=df_merge_web_clean['price'], kde=True, ax=axes[1])
axes[0].set(xlabel='')
plt.suptitle("Distribution des prix (avec repérage des outliers)")
plt.show()
```

```
python

seaborn as sns
matplotlib.pyplot as plt

es = plt.subplots(2, 1, figsize=(8,5), sharex=True, gridspec_kw={'height_ratios': [1,3]})
plot(x=df_merge_web_clean['price'], ax=axes[0])
plot(x=df_merge_web_clean['price'], kde=True, ax=axes[1])
set(xlabel='')
title("Distribution des prix (avec repérage des outliers)")
()
```

Résultat obtenu



Comparaison des options

Critère	Option A — plotly (histogram + marginal box)	Option B — seaborn (boxplot + histplot)
Qualité	Répartition + prix anormaux sur un seul graphique, interactif (zoom, survol)	Même info, mais en image fixe
Temps de mise en œuvre	1 ligne de code	~6 lignes
Reproductibilité	Aucun nouvel outil à installer (Plotly déjà utilisé)	Nouvel outil à installer (Seaborn)
Cohérence avec l'existant	Cohérent avec le reste du projet (déjà en Plotly)	Change de style visuel

Mon choix : Option A

Même information utile que la version Seaborn, avec moins de code, sans outil à installer, et en gardant le style visuel du reste du projet (déjà en Plotly).

1. Effectuer une veille métier et technologique

Besoin de veille

Dans le notebook précédent, l'exploration de chaque fichier (ventes ERP, ventes web, table de correspondance) était faite à la main et répétée pour chacun : taille des données, types de colonnes, doublons, valeurs manquantes, statistiques de base. Besoin : automatiser cette première exploration (« EDA ») en une seule commande.

Panel de solutions

- Méthode initiale, utilisée comme point de comparaison
- **Option 1 : ydata-profiling**

Rapport HTML complet en quelques lignes de code : statistiques, valeurs manquantes, corrélations, alertes qualité.

- **Option 2 : Sweetviz**

Rapport plus court, centré sur les liens entre colonnes ; particulièrement adapté pour comparer deux jeux de données (ex : avant/après nettoyage).

Sources fiables identifiées

- Documentation officielle ydata-profiling : docs.profiling.ydata.ai (consulté le 20/07/2026)
- Dépôt GitHub officiel ydata-profiling : github.com/ydataai/ydata-profiling (consulté le 20/07/2026)
- Dépôt GitHub officiel Sweetviz : github.com/fbdesignpro/sweetviz (consulté le 20/07/2026)

Automatisation de la veille

Abonnement aux notifications GitHub de ydata-profiling et Sweetviz : toute nouvelle version ou changement important est signalé automatiquement, sans revérification manuelle.

Comparatif des solutions

Critère	ydata-profiling	Sweetviz
Cas d'usage	Exploration approfondie d'un seul dataset	Comparaison de deux datasets ou analyse d'une variable cible
Qualité	Rapport très détaillé (corrélations, interactions, alertes qualité)	Rapport plus visuel mais moins exhaustif
Robustesse / biais	Purement descriptif, aucun calcul qui pourrait fausser les résultats	Idem
Coût en temps	Génération plus lente	Génération plus rapide
Reproductibilité	100 % local, résultat fiable, une ligne de code	Idem
Sécurité / conformité	Aucune donnée envoyée à l'extérieur	Idem
Sobriété / impact	100 % local, pas d'appel à un service en ligne	Idem
Maintenabilité	Outil établi, large communauté, bien documenté	Outil actif, communauté plus restreinte

Critère	ydata-profiling	Sweetviz
Prérequis	pip install ydata-profiling	pip install sweetviz
Avantages	Vue complète et détaillée en une commande	Rapport léger et rapide à générer
Limites	Plus lent sur de gros volumes, rapport parfois dense	Moins détaillé, moins pertinent pour explorer un seul dataset en profondeur

Décision : Le choix s'est porté sur ydata-profiling. Cet outil est le plus adapté, car le projet consiste à analyser chaque fichier séparément, sans comparer plusieurs jeux de données.

2. Identifier les besoins métier et formaliser un cahier des charges fonctionnel

Notebook destiné avant tout aux équipes métier (non techniques).

Contexte

Bottleneck, société de vente de vin en ligne. Trois équipes utilisent cette analyse :

- Commercial / direction : CA et classement des ventes, pour décider quels produits promouvoir.
- Achats / stocks : mois de stock, pour prioriser le réapprovisionnement.
- Finance : prix et marges, pour valider la politique tarifaire et détecter les anomalies.

Environnement changeant : le catalogue évolue régulièrement (nouveaux millésimes, fournisseurs), prix et stocks sont mis à jour en continu. L'analyse doit rester reproductible sur un nouvel export de données.

Priorités du projet

- Priorité 1 : fiabiliser le chiffre d'affaires (fusion ERP / web correcte, sans perte ni doublon).
- Priorité 2 : détecter les anomalies de prix (outliers) pour objectiver la politique tarifaire.
- Priorité 3 : objectiver les besoins de réapprovisionnement à partir des mois de stock.

Problématique métier reformulée

Comment fiabiliser les données ERP / web pour calculer un CA fiable, identifier les articles performants, détecter les anomalies de prix et objectiver le réapprovisionnement ?

Périmètre

- Traité : les 3 sources de données, répartition des prix, CA par article, classement des ventes, taux de marge, détection des prix anormaux.
- Non traité : pas de prévision des ventes, pas de données clients individuelles, pas de ventes hors site web.

Contraintes

- Données manquantes : référence produit ou identifiant web absents sur certaines lignes → supprimées après vérification (part infime du catalogue).
- Qualité : incohérences stock corrigées ; doublons vérifiés.
- RGPD : uniquement des données produits, aucune donnée personnelle → enjeu très faible.
- Volumétrie : faible (~800 lignes côté ERP), pas de contrainte de performance.
- Outillage imposé : Python / Jupyter / Pandas / Plotly.
- Budget : nul — uniquement des outils open source.
- Délais : pas de contrainte temps réel ; une seule itération (voir Partie 4).

Critères de réussite / KPI

Catégorie	Critères de réussite / KPI
Data	Taux de données manquantes documenté ; 0 doublon après nettoyage.
Insights	CA total cohérent (outliers < 5 % du catalogue) ; taux de marge interprétable.
Opérationnel	Notebook exécutable sans erreur ; temps de calcul raisonnable ; graphiques lisibles.

Mini-plan de formation pour les utilisateurs métier

Public cible : équipe métier chez Bottleneck (non-développeurs).

- Prise en main de Jupyter (30 min) : ouvrir, exécuter une cellule, modifier un chemin de fichier.
- Lecture des rapports ydata-profiling (30 min) : alertes qualité, valeurs manquantes, corrélations.
- Lecture des graphiques Plotly (15 min) : zoom, survol, exploration du classement des ventes.

Accessibilité : support écrit (captures commentées) en complément de la session orale, consultable à son rythme.

Arbitrage

Formation courte et pratique (< 1h15) plutôt qu'une formation Python complète : l'objectif est que les métiers exploitent le notebook, pas qu'ils le développent.

3. Organiser un projet d'analyse data à l'aide des outils de gestion de projet

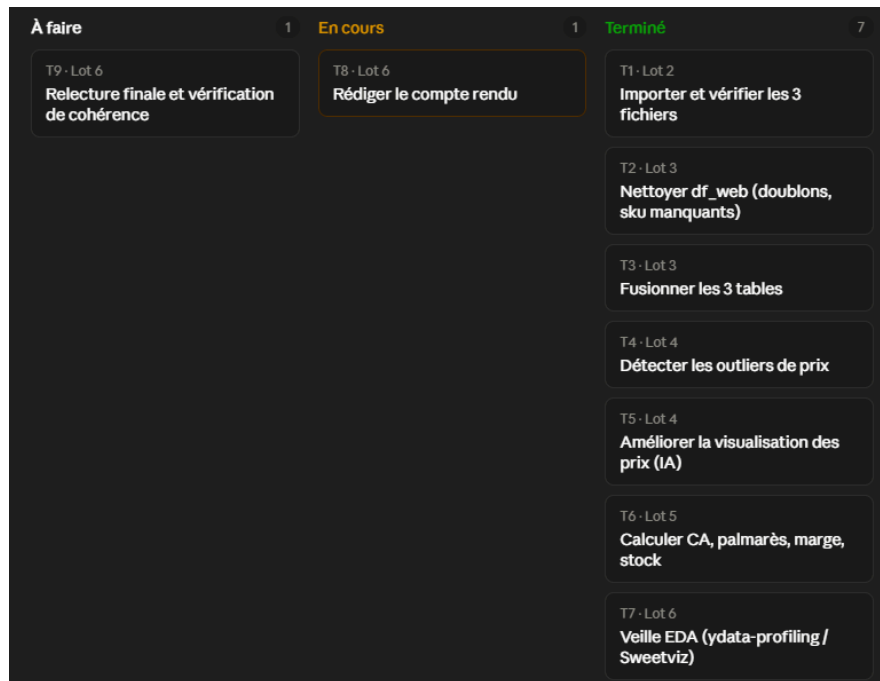
Découpage en lots

Lot	Contenu	Correspond à
Lot 0 — Setup	Installation de l'environnement (Python, pandas, plotly, ydata-profiling, sweetviz), vérification que le notebook s'exécute sans erreur	Environnement (voir README)
Lot 1 — Cadrage	Définir la problématique métier, les parties prenantes, le périmètre	Partie 3 (cahier des charges)
Lot 2 — Data audit	Import des 3 fichiers, vérification qualité (doublons, NaN, incohérences)	Étape 1-2 du notebook
Lot 3 — Nettoyage et fusion	Correction des incohérences, fusion des 3 tables	Étape 3
Lot 4 — EDA / analyse des prix	Détection des outliers, visualisation de la distribution	Étape 4
Lot 5 — Analyse métier	Calcul du CA, palmarès, taux de marge, mois de stock	Étape 5
Lot 6 — Restitution	Compte rendu, notebook nettoyé, veille documentée	Ce document

Backlog, planning et points de contrôle

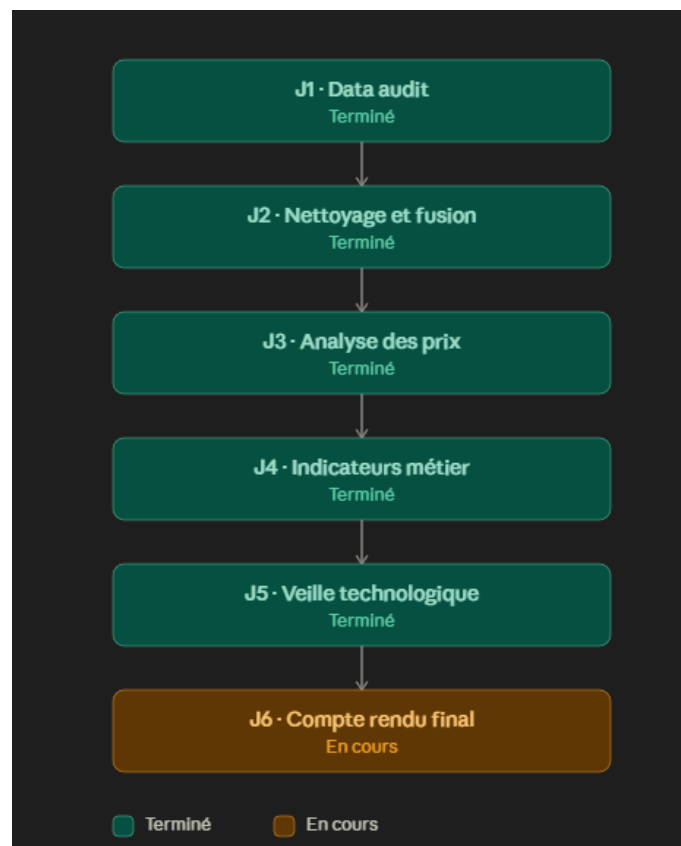
Backlog de 10 tâches suivi sur Kanban Notion (À faire / En cours / Terminé), avec estimation (S/M/L), dépendances et Definition of Done pour chacune. Points clés :

- **T0 — Installer l'environnement** (S, aucune dépendance) : Definition of Done = pip install exécuté sans erreur, notebook ouvrable dans Jupyter.
- Tâches critiques suivantes : T2 (nettoyage web, dépend de T1), T3 (fusion, dépend de T2), T4 (détection outliers, dépend de T3).
- Definition of Done systématique : ex. "0 doublon, 0 référence manquante" (T2), "fusion sans perte de lignes injustifiée" (T3).



Suivi du backlog (Kanban Notion)

Planning : 7 jalons (J0 à J6) — J0 = environnement prêt, avant même le data audit (J1).



Timeline des jalons

Points de contrôle : À chaque fois qu'une étape importante du projet est terminée (la fin d'un lot, ou un jalon clé comme "data audit terminé" ou "fusion des données terminée"), on enregistre l'état du travail avec un petit outil appelé Git, en lui donnant un message court qui décrit ce qui vient d'être fait, par exemple "vérification qualité des 3 fichiers terminée". Cela crée un point de sauvegarde daté et identifié, qu'on peut ensuite retrouver et consulter à tout moment, dans l'ordre chronologique. Pour les étapes vraiment marquantes du planning, on ajoute en plus un repère spécial qui permet de revenir directement à cet instant précis du projet, sans avoir à chercher dans une liste de fichiers numérotés à la main. L'ensemble de ces sauvegardes forme un historique complet et consultable en ligne, qui sert de preuve concrète du suivi du projet au fil du temps.

Gestion des risques

Risque	Probabilité	Impact	Mitigation
Environnement mal configuré (version Python/libs incompatible)	Faible	Moyen	Fichier requirements.txt versionné, testé sur un environnement propre avant restitution
Perte de lignes lors de la fusion des 3 tables	Moyenne	Élevé	Vérifier le nombre de lignes avant/après chaque fusion
Outliers de prix mal filtrés (seuil IQR trop strict/large)	Moyenne	Moyen	Comparer 2 méthodes (z-score vs IQR), documenter le choix
Données manquantes non traitées (sku, id_web)	Faible	Moyen	Quantifier le taux de lignes supprimées, vérifier qu'il reste marginal

Glossaire

Quelques termes techniques utilisés dans ce document, expliqués simplement.

Terme	Explication
Notebook	Document interactif (utilisé avec l'outil Jupyter) qui mélange du code, des résultats et des explications, exécuté par blocs.
Dataset / jeu de données	Un ensemble de données organisées, ici sous forme de tableaux (ventes, stock, prix...).
EDA (exploration de données)	La première étape d'une analyse : regarder les données pour comprendre leur structure, leurs erreurs et leurs tendances avant de les exploiter.
Boxplot / boîte à moustaches	Un graphique qui résume rapidement une série de valeurs à l'aide d'un rectangle (où se trouve la majorité des valeurs) et de traits (les valeurs extrêmes).
Outlier / valeur / prix anormal	Une donnée très différente des autres (par exemple un prix beaucoup plus élevé que la moyenne), qui peut être une erreur ou un cas réellement particulier.
Z-score / IQR	Deux méthodes de calcul différentes pour repérer automatiquement les valeurs anormales dans une série de chiffres.
ERP	Le logiciel de gestion interne de l'entreprise (stocks, ventes, produits), par opposition aux données du site web.
SKU	Le code de référence unique d'un produit, qui permet de le retrouver dans les différents fichiers.

Terme	Explication
CA (chiffre d'affaires)	Le montant total des ventes réalisées.
KPI	Un indicateur chiffré utilisé pour mesurer si un objectif est atteint (ex. taux de doublons, temps d'exécution).
RGPD	La réglementation européenne qui encadre la protection des données personnelles.
Kanban	Un tableau de suivi visuel des tâches, organisé en colonnes (par exemple À faire / En cours / Terminé).
Backlog	La liste de toutes les tâches à réaliser sur un projet, généralement classées par priorité.
Jalon	Une étape clé du projet, avec une date et un contenu attendu, qui permet de suivre l'avancement.
Itération / sprint	Un cycle de travail court et répété, utilisé dans certaines méthodes de gestion de projet pour avancer par petites étapes.